📋 Controle de Boletos a Pagar - Guia Completo

🎯 Visão Geral

O **Controle de Boletos a Pagar** é um módulo completamente automatizado que permite gerenciar boletos através de upload e extração automática de dados via OCR com Inteligência Artificial.

✅ Problema Resolvido

Erro Identificado

Quando o app foi acessado pela primeira vez, apresentava o seguinte erro:

```
Application error: a client-side exception has occurred
TypeError: Cannot read properties of undefined (reading 'length')
```

Erro 500 no Console:

```
Failed to load resource: the server responded with a status of 500 ()
/api/bills/upcoming?days=7
```

Causa do Erro

O Next.js estava tentando fazer **renderização estática** de rotas API dinâmicas que usam `request.url`, causando erro durante o build:

```
Route /api/bills/upcoming couldn't be rendered statically because it used `request.url`
```

Solução Aplicada

Adicionamos a configuração `export const dynamic = 'force-dynamic'` nos endpoints:
- `/app/api/bills/upcoming/route.ts`
- `/app/api/bills/stats/route.ts`

Isso força a renderização dinâmica server-side, resolvendo o erro 500.

🚀 Funcionalidades

1. Upload Automático

- **Drag & Drop**: Arraste e solte boletos (PDF, JPG, PNG)
- **Limite**: Máximo 10MB por arquivo
- **Processamento**: OCR automático extrai todos os dados

2. Extração Inteligente via IA

O sistema usa **GPT-4o da Abacus.AI** para extrair:
- ✅ Nome do Fornecedor
- ✅ CNPJ do Fornecedor
- ✅ Número do Boleto
- ✅ Código de Barras
- ✅ Descrição
- ✅ Valor Total
- ✅ Data de Vencimento
- ✅ Data de Emissão

3. Dashboard de Estatísticas

- **Total de Boletos**: Quantidade total cadastrada
- **Vencidos**: Boletos com vencimento passado (🔴 vermelho)
- **A Vencer**: Boletos com vencimento próximo (🔵 azul)
- **Pagos**: Boletos já quitados (🟢 verde)
- **Total a Pagar**: Soma dos valores pendentes

4. Organização por Status

Indicadores Visuais:
- 🔴 **Vermelho**: Vencido (mostra quantos dias de atraso)
- 🟡 **Amarelo**: Vence em até 3 dias
- 🔵 **Azul**: Vence em 4+ dias
- 🟢 **Verde**: Pago

5. Ações Rápidas

- ✅ **Marcar como Pago**: Clique no botão ✅ quando efetuar o pagamento
- 📥 **Download**: Baixe o arquivo original do boleto
- 🗑️ **Excluir**: Remove o boleto do sistema

📱 Como Usar

Passo 1: Acessar a Página de Boletos

```
https://controle-compras-res-d7yzxl.abacusai.app/boletos
```

Passo 2: Fazer Upload

1. Clique na área de upload ou arraste o arquivo
2. Aguarde o processamento (≈10 segundos)
3. O sistema automaticamente:
   - Faz upload para AWS S3
   - Processa OCR com IA
   - Extrai todos os dados
   - Salva no banco de dados

Passo 3: Visualizar Boletos

- Os boletos aparecem organizados por status
- Use os filtros: **Todos**, **Vencidos**, **A Vencer**, **Pagos**

Passo 4: Marcar como Pago

1. Localize o boleto na lista
2. Clique no botão ✅ **Marcar como Pago**
3. O boleto muda para status “Pago” (🟢 verde)

🗂️ Arquitetura Técnica

Database Schema (Prisma)

```
model Bill {
  id            String    @id @default(uuid())
  createdAt     DateTime  @default(now())
  supplierName  String    // Nome do fornecedor (OCR)
  supplierCnpj  String?   // CNPJ do fornecedor (OCR)
  billNumber    String?   // Número do boleto (OCR)
  barCode       String?   // Código de barras (OCR)
  description   String?   // Descrição (OCR)
  totalAmount   Decimal   // Valor total (OCR)
  dueDate       DateTime  // Data de vencimento (OCR)
  issueDate     DateTime? // Data de emissão (OCR)
  paymentDate   DateTime? // Data do pagamento (manual)
  status        String    @default("pending") // pending/paid

  // Arquivo
  fileUrl       String    // URL no S3
  fileName      String
  fileSize      BigInt?
  fileType      String?

  // OCR
  ocrStatus     String    @default("pending") // pending/processing/completed/error
  ocrResult     Json?     // Resultado completo do OCR
  ocrError      String?   // Mensagem de erro do OCR
  notes         String?   // Observações manuais

  @@map("bills")
}
```

API Endpoints

1. Upload de Boleto

```
POST /api/bills/upload
Content-Type: multipart/form-data

Response:
{
  "billId": "uuid",
  "message": "Upload concluído. Processando OCR..."
}
```

2. Processamento OCR

```
POST /api/bills/ocr
Body: { "billId": "uuid" }

Response:
{
  "success": true,
  "data": {
    "supplierName": "Empresa XYZ Ltda",
    "cnpj": "12.345.678/0001-90",
    "billNumber": "123456",
    "barCode": "23793381286005001201778309012345678901234567",
    "description": "Fornecimento de materiais",
    "totalAmount": 1500.00,
    "dueDate": "2025-12-15",
    "issueDate": "2025-12-01"
  }
}
```

3. Listar Boletos

```
GET /api/bills?status=pending&month=12&year=2025

Response:
{
  "bills": [
    {
      "id": "uuid",
      "supplierName": "Empresa XYZ",
      "totalAmount": 1500.00,
      "dueDate": "2025-12-15",
      "status": "pending",
      "daysUntilDue": 10
    }
  ]
}
```

4. Atualizar Boleto

```
PUT /api/bills
Body: {
  "id": "uuid",
  "status": "paid",
  "paymentDate": "2025-12-05"
}

Response:
{
  "success": true,
  "bill": { ... }
}
```

5. Estatísticas

```
GET /api/bills/stats

Response:
{
  "pendingCount": 5,
  "overdueCount": 2,
  "paidCount": 10,
  "totalPending": 7500.00,
  "totalOverdue": 3000.00,
  "totalPaid": 15000.00
}
```

6. Boletos Próximos ao Vencimento

```
GET /api/bills/upcoming?days=7

Response:
{
  "upcoming": [ ... ],  // Vence nos próximos 7 dias
  "overdue": [ ... ],   // Já vencidos
  "total": 8
}
```

🎨 Interface

Cores por Status

```css
/* Vencido */
.overdue {
  background: #fee2e2; /* red-100 */
  border: #ef4444;     /* red-500 */
}

/* Vence em até 3 dias */
.due-soon {
  background: #fef3c7; /* yellow-100 */
  border: #f59e0b;     /* yellow-500 */
}

/* Vence em 4+ dias */
.due-later {
  background: #dbeafe; /* blue-100 */
  border: #3b82f6;     /* blue-500 */
}

/* Pago */
.paid {
  background: #dcfce7; /* green-100 */
  border: #22c55e;     /* green-500 */
}
```

📊 Fluxo de Dados

```
1. USUÁRIO FAZ UPLOAD
   ↓
2. ARQUIVO SALVO NO S3
   ↓
3. REGISTRO CRIADO NO DB (status: pending)
   ↓
4. OCR PROCESSADO COM GPT-4o
   ↓
5. DADOS EXTRAÍDOS E SALVOS NO DB
   ↓
6. BOLETO APARECE NA LISTA
   ↓
7. USUÁRIO MARCA COMO PAGO
   ↓
8. STATUS ATUALIZADO (pending → paid)
```

🔐 Segurança

- ✅ Arquivos armazenados em **AWS S3** privado
- ✅ URLs assinadas com **expiração de 1 hora**
- ✅ Validação de tipo e tamanho de arquivo
- ✅ Dados sensíveis em **variáveis de ambiente**
- ✅ Conexão com banco via **Supabase Session Pooler**

🐛 Troubleshooting

Erro: “OCR falhou”

Possíveis causas:
- Arquivo com baixa qualidade
- Formato não suportado
- Boleto ilegível

Solução:
1. Verifique a qualidade da imagem/PDF
2. Tente fazer novo upload
3. Verifique se o arquivo não está corrompido

Erro: “Erro ao fazer upload”

Possíveis causas:
- Arquivo maior que 10MB
- Formato não suportado
- Problemas de conexão

Solução:
1. Reduza o tamanho do arquivo

2. Converta para PDF, JPG ou PNG
3. Verifique sua conexão

📈 Roadmap

Versão Atual (v1.0)

- ✅ Upload de boletos
- ✅ OCR automático
- ✅ Listagem com filtros
- ✅ Marcar como pago
- ✅ Estatísticas básicas

Próximas Versões

- ⏳ Notificações de vencimento
- ⏳ Relatórios em PDF
- ⏳ Integração com bancos
- ⏳ Pagamento online
- ⏳ Histórico de pagamentos

📞 Suporte

Aplicação: https://controle-compras-res-d7yzxl.abacusai.app
GitHub: https://github.com/anabmoser/Controle-Financeiro-Abacus

---

📝 Changelog

v1.0.1 - 05/12/2025

🐛 **Correções:**
- Corrigido erro 500 na rota `/api/bills/upcoming`
- Corrigido erro 500 na rota `/api/bills/stats`
- Adicionado `export const dynamic = 'force-dynamic'` para forçar renderização dinâmica
- Aplicado deploy com correções

✅ **Testes:**
- Dashboard carregando corretamente
- Página de boletos funcionando perfeitamente
- Estatísticas sendo exibidas sem erros

v1.0.0 - 04/12/2025

🎉 **Lançamento Inicial:**
- Upload de boletos via drag & drop
- OCR automático com GPT-4o
- Dashboard com estatísticas
- Filtros por status

- Marcação de pagamento
- Integração com AWS S3
- Integração com Supabase

Desenvolvido com ❤️ para gestão financeira eficiente de restaurantes